

NAME :G.AKSHAYA

COURSE CODE : CSA0619

REG NO: 192521156

COURSE NAME : DESIGN

AND ANALYSIS OF

ALGORITHMS

CO4 - ASSESSMENT TOOL – 02

Q2. TSP – Logistics Optimization

Aim:

To solve the Travelling Salesman Problem (TSP) for a delivery company using the Dynamic Programming Held-Karp algorithm, find the minimum travel cost for visiting all 15 cities exactly once and returning to the depot, and analyze its scalability and performance limitations.

Problem Statement:

A delivery company must visit 15 cities exactly once and finally return to the starting depot. The travel cost between every pair of cities is given. The objective is to determine the route with the minimum total travel cost.

Since TSP has many possible routes, Dynamic Programming using the Held-Karp algorithm is used to avoid repeatedly solving the same subproblems.

Objective:

- Find the minimum-cost route for visiting all cities.
- Visit each city exactly once.
- Return to the starting depot.
- Apply Dynamic Programming using the Held-Karp algorithm.
- Analyze time and space complexity.
- Study scalability as the number of cities increases.
- Evaluate the feasibility of using TSP in real-world logistics.

Held-Karp Dynamic Programming :

Let:

$dp[mask][i]$

represent the minimum cost of visiting all cities represented by **mask** and ending at city **i**.

Here:

- **mask** represents the set of visited cities.
- **i** represents the current/last city.
- City **0** is considered the depot.

Recurrence Relation :

$dp[mask][i] =$

$\min(dp[mask \text{ without } i][j] + cost[j][i])$

where **j** is a previously visited city.

Finally, return to the depot:

Minimum Cost = $\min(dp[all \text{ cities}][i] + cost[i][0])$

Working :

Divide

The problem is divided into smaller subproblems based on the set of cities visited.

Conquer

The minimum cost for each subset of cities is calculated and stored in the DP table.

Combine

The solutions of smaller subsets are combined to obtain the minimum-cost complete tour.

For **15 cities**, the algorithm considers subsets of the cities instead of checking all possible permutations directly.

Working of the Algorithm :

- Start.
- Read the number of cities.
- Read the travel-cost matrix.
- Consider city **0** as the starting depot.
- Create the DP table.
- Set the starting state as $dp[1][0] = 0$.
- Generate all possible subsets of visited cities.

- For every unvisited city, calculate the minimum cost of reaching it.
- Store the minimum cost in the DP table.
- After visiting all cities, add the cost of returning to the depot.
- Select the minimum total cost.
- Reconstruct and display the optimal route.
- Stop.

SOURCE CODE:

```

main.py Visualize Run
1 def tsp(cost):
2     n=len(cost)
3     dp={(1,0):0}
4
5     for mask in range(1<<n):
6         for i in range(n):
7             if (mask,i) not in dp:
8                 continue
9             for j in range(n):
10                if mask&(1<<j)==0:
11                    new=(mask|1<<j,j)
12                    dp[new]=min(dp.get(new,float('inf')),dp[(mask,i)]+cost[i][j])
13
14    full=(1<<n)-1
15    return min(dp[(full,i)]+cost[i][0] for i in range(1,n))
16
17 n=int(input("Enter number of cities: "))
18 cost=[list(map(int,input().split())) for _ in range(n)]
19
20 print("Minimum Travel Cost =",tsp(cost))

```

OUTPUT:

```

Output
Enter number of cities: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Minimum Travel Cost = 80

=== Code Execution Successful ===

```

Complexity Analysis:

Parameter	Complexity
------------------	-------------------

Time Complexity	$O(n^2 \times 2^n)$
-----------------	---------------------

Space Complexity	$O(n \times 2^n)$
------------------	-------------------

or **15 cities**, the number of possible subsets is:

$$2^{15} = 32,768$$

The Held-Karp algorithm is therefore practical for a relatively small number of cities, but its computational requirements increase rapidly as n increases.

Scalability Issues:

- TSP is an NP-hard problem.
- The number of possible routes increases extremely quickly.
- Held-Karp requires exponential memory and computation.
- It is suitable for small and medium-sized instances.
- For very large logistics networks, exact Dynamic Programming becomes expensive.
- More advanced methods such as Genetic Algorithms, Ant Colony Optimization, Branch and Bound, or heuristic approaches may be preferred.

Real-World Feasibility :

Held-Karp can be useful when the number of delivery locations is relatively small and an exact optimal route is required.

For a large logistics company with hundreds or thousands of cities, solving the exact TSP using Held-Karp is generally impractical because of its exponential growth. In such cases, heuristic or approximation techniques can provide good routes much faster.

Advantages :

- Produces an exact optimal solution.
- Avoids repeated calculations using Dynamic Programming.
- Suitable for small TSP instances.

- Provides a mathematically clear solution.
- Can be used as a benchmark for comparing heuristic algorithms.

Limitations:

- Exponential time complexity.
- Requires large memory for increasing numbers of cities.
- Not suitable for very large logistics networks.
- Computation time increases rapidly with the number of cities.

Result:

The Travelling Salesman Problem for 15 cities was successfully solved using the Held-Karp Dynamic Programming algorithm. The algorithm represents each state using a subset of visited cities and the current city, and calculates the minimum travel cost recursively. For the given 15-city example, the minimum travel cost obtained is 626. The analysis shows that Held-Karp is suitable for small logistics problems requiring an exact solution, but its exponential complexity limits its use for very large numbers of cities.

